

Reviewer Pack — Plethysm Multiplicity Gap in Symmetric Powers of Permanent v...

Entry ee4cd4f186fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 13:26:18 UTC

Plethysm Multiplicity Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: ee4cd4f186fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 13:26:18 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality, plethysm, algebraic combinatorics **Field B** (complexity object): Algebraic complexity theory (Extended Frege / Frege lower bounds)

Statement:

For all $n \geq 2$ and $m < n^{\{1.5\}}$, the multiplicity of the trivial representation in the decomposition of $S^m(\text{perm}_n)$ under $GL(n)$ exceeds that in $S^{m(\det_m O(1))}$ by a factor of $\Omega(n^{\{1/3\}})$

Rationale (proposer’s reasoning):

Plethysm multiplicities capture orbit closure invariants under linear substitutions. If perm_n ’s decomposition contains exponentially more trivial components than $\det_m O(1)$ for $m < n^{\{1.5\}}$, this would separate their orbit closures and imply super-polynomial EF lower bounds via Landsberg’s tensor rank framework.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d6fe79b3612d6141

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

    def binomial_coefficient(n, k):
        return factorial(n) // (factorial(k) * factorial(n - k))

    def plethysm(m, n):
        if m == 0:
            return 1
        result = 0
        for k in range(1, m + 1):
            result += binomial_coefficient(m, k) * plethysm(k, n)
        return result

    def trivial_multiplicity(m, n):
        return plethysm(m, n)

    def det_multiplicity(m, n):
        if m == 0:
            return 1
        result = 0
        for k in range(1, m + 1):
            result += binomial_coefficient(m, k) * det_multiplicity(k - 1, n)
        return result

    def compute_multiplicities(n, m_values):
        perm_multiplicities = [trivial_multiplicity(m, n) for m in m_values]
        det_multiplicities = [det_multiplicity(m, n) for m in m_values]
```

```

    return perm_multiplicities, det_multiplicities

n_values = [5, 10, 15, 20, 30, 40]
m_values = [int(n**1.5 * random.random()) for _ in range(10) for n in n_values]

perm_multiplicities, det_multiplicities = compute_multiplicities(max(n_values), m_values)

mean_perm = sum(perm_multiplicities) / len(perm_multiplicities)
mean_det = sum(det_multiplicities) / len(det_multiplicities)

if mean_perm <= mean_det:
    return {
        "metric_name": "Multiplicity Gap",
        "metric_value": mean_perm - mean_det,
        "instances_tested": len(m_values),
        "conjecture_holds": False,
        "counterexample": "multiplicity_gap_not_met"
    }

return {
    "metric_name": "Multiplicity Gap",
    "metric_value": mean_perm - mean_det,
    "instances_tested": len(m_values),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={seed}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 87, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 59, in run_trial
    perm_multiplicities, det_multiplicities = compute_multiplicities(max(n_values), m_values)
                                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 52, in compute_multi
    perm_multiplicities = [trivial_multiplicity(m, n) for m in m_values]
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 41, in trivial_multi
    return plethysm(m, n)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 37, in plethysm
    result += binomial_coefficient(m, k) * plethysm(k, n)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 37, in plethysm
    result += binomial_coefficient(m, k) * plethysm(k, n)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 37, in plethysm
    result += binomial_coefficient(m, k) * plethysm(k, n)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 992 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38989d51.py", line 30, in binomial_coef
    return factorial(n) // (factorial(k) * factorial(n - k))
           ^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to recursion depth limits, preventing empirical validation of the conjecture | next: Optimize plethysm computation with memoization and iterative approaches to handle large n/m values

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	115049
2	propose	ollama_remote	qwen3:8b	0	0	47361
3	preregistration	ollama_remote	qwen3:8b	0	0	24148

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	qwen3:8b	0	0	20654
5	novelty	ollama_remote	qwen3:8b	0	0	18867
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39162
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9882
8	judge	ollama_remote	qwen3:8b	0	0	20027

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 295149 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/ee4cd4f186fc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ee4cd4f186fc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ee4cd4f186fc.tar.gz (if generated)